



POLITECNICO
MILANO 1863

BEST BIKE PATHS

ATD

Filippo Raimondi

Matteo Sala

Deliverable: ATD

Title: Acceptance Test Deliverable

Authors: Raimondi Filippo, Sala Matteo

Version: 1.0

Date: 07-02-2026

Download page: <https://github.com/m-tteo/sw-eng-2-raimondi-sala>

Copyright: Copyright © 2026, Raimondi Filippo, Sala Matteo – All rights reserved

Contents

Table of Contents	2
1 Analyzed Project	3
1.1 Authors	3
1.2 Repository	3
1.3 Reference Documents	3
2 Installation setup	4
2.1 Documentation Inconsistencies	4
2.2 PostgreSQL	4
2.3 Public Deployment	4
3 Acceptance Test Cases	5
3.1 Authentication	5
3.1.1 Registration and Login	5
3.2 Views and Functionalities	5
3.2.1 Search (/search)	5
3.2.2 Manual Path Insertion (/path/manual)	5
3.2.3 Trip Dashboard (/trip/dashboard)	5
3.2.4 Trip History (/trip/history)	6
3.2.5 Profile (/profile)	6
4 Additional Comments	7
4.1 Documentation Quality	7
4.1.1 RASD	7
4.1.2 DD	7
4.1.3 ITD	7
4.2 Code Quality	7
5 Use of Generative AI	8
6 Effort Spent	9
References	10

1 Analyzed Project

1.1 Authors

The authors of the analyzed project are:

- Airoidi Michelangelo
- Nava Francesco

1.2 Repository

The repository of the analyzed project is available [here](#).

1.3 Reference Documents

1. A.Y. 2025-2026 Software Engineering 2 - Requirement Engineering and Design Project: goal, schedule, and rules [\[2\]](#)
2. A.Y. 2025-2026 Software Engineering 2 - I&T assignment goal, schedule, and rules [\[1\]](#)
3. Best Bike Path (BBP) Requirements Analysis And Specification Document (RASD), version 4 [\[5\]](#)
4. Best Bike Path (BBP) Design Document (DD), version 2 [\[3\]](#)
5. Best Bike Path (BBP) Implementation and Test Deliverable Document (ITD), version 1 [\[4\]](#)

2 Installation setup

To install the prototype we followed both the guide in the READMEs of GitHub and the one present in the ITD document. We downloaded the *I&T* folder from GitHub, made and activated the Python virtual environment for every service present on different shells, installed all the requirements for every service, for the path-management, trip-management and user-management set up the database and launched each of the 5 services.

2.1 Documentation Inconsistencies

There are two different installation references: the one present in the README files in each microservice folder and one in the ITD document, the one in the ITD is clearer but presents some discrepancies as we proceed to explain.

Firstly, the instructions regarding the environment variables in the ITD are ambiguous; the document implies that every microservice should share the same `.env` file as stated in Section 6.2 "Each microservice requires the following environment variables configured in a `.env` file:" however this is not true, as some services will get this error: "Extra inputs are not permitted", so we copied the environment variables present in the README and some provided by email from the project's authors, because in the README of the frontend service there are no environment variables present.

Secondly, there is a discrepancy regarding the API-Gateway configuration. The ITD specifies that the API-Gateway listens on port 8003, whereas the README indicates port 8080. Attempts to configure the system according to the ITD (port 8003) failed; the system appears to function correctly only when the API-Gateway is set to port 8080 as per the README.

2.2 PostgreSQL

The guidelines for setting up the PostgreSQL database are absent, therefore an inexperienced user could struggle as there is no explanation on setting up the user and the database and an explanation of how to get the Supabase database URL and password is not present in the documentation.

2.3 Public Deployment

A notable positive aspect is that the project provides a publicly accessible deployment on Railway (<https://bbp-frontend-service-production.up.railway.app>). This allows immediate testing of the prototype without requiring local installation, significantly simplifying the acceptance testing process.

3 Acceptance Test Cases

The following sections detail the acceptance tests performed on the main views and functionalities of the application. The tests focus on usability, adherence to documentation, and compliance with the assignment requirements.

Tests were selected based on the implemented requirements documented in the ITD (R.1-R.7, R.9-R.16, R.20-R.24, R.38-R.41), prioritizing core user workflows and edge cases to verify system robustness. Features excluded from implementation were not tested.

3.1 Authentication

The authentication flow was tested for both registration and login procedures (Requirements R.38-R.41).

3.1.1 Registration and Login

- **Registration:** Tests verified email uniqueness enforcement (duplicate emails rejected), password field validation, and successful account creation. Error messages for duplicate emails lack clarity.
- **Login:** Tests verified correct credential validation (valid/invalid combinations) and session token generation. Error messages lack clarity and specificity.

3.2 Views and Functionalities

3.2.1 Search (/search)

The search functionality operates exactly as described in the provided documentation (Requirements R.1-R.5). Specific edge cases we tested:

- Insertion of non-existent street names.
- Selection of Origin and Destination points between which no viable path exists.

In all tested scenarios, the system handled the inputs correctly without crashing or entering an infinite loop. Furthermore, the service correctly identifies the optimal path. The score calculation mechanism operates exactly as described in the documentation.

3.2.2 Manual Path Insertion (/path/manual)

The manual path insertion functionality was found to be intuitive and user-friendly (Requirements R.15-R.16, R.20-R.24). We conducted specific tests to verify the robustness of this view, including:

- Placing obstacles outside the relevant path.
- Inserting extremely long paths or extremely short paths.

In all these cases, the system functioned correctly without throwing errors. The only failed test, although this edge case is not critical for the platform's correctness, is that a path of length 0m could be inserted by placing two points on top of the other.

3.2.3 Trip Dashboard (/trip/dashboard)

The functionality is implemented in accordance with the project documentation (Requirements R.6-R.7, R.9), allowing users to start trip recording, track GPS coordinates, and stop when completed.

3.2.4 Trip History (/trip/history)

The view functions correctly and displays all required information such as distance traveled, start and end timestamps, duration, and weather information (Requirements R.10-R.14). We tested the following:

- **Access Control:** Attempts to access a non-existent trip ID resulted in an appropriate error. Furthermore, attempts to access a private trip belonging to another user were successfully blocked, showing no data.
- **Deletion:** The functionality to delete a trip was tested; the trip was correctly removed from both the visualization and the database.

3.2.5 Profile (/profile)

This section is minimalistic, displaying only the information of the currently logged-in user. While the "Log out" functionality works as expected, the profile lacks essential data management features. Specifically:

- It is not possible to modify user data.
- It is not possible to delete the account.

These missing features conflict with RASD Constraint C.5 regarding GDPR compliance, which requires user rights to data modification and deletion.

4 Additional Comments

4.1 Documentation Quality

4.1.1 RASD

The Requirements Analysis and Specification Document demonstrates thorough requirements engineering. It provides well-defined functional requirements with clear priority levels and goal mappings and the UML diagrams are comprehensive.

4.1.2 DD

The Design Document details an extremely robust architecture. The API endpoint documentation is precise, specifying authentication requirements, request/response schemas, and error handling for each endpoint. The requirements traceability section effectively maps RASD requirements to architectural components.

4.1.3 ITD

The Implementation and Test Deliverable transparently documents the deviations from the DD, clearly explaining the reasons for technology stack changes and infrastructure simplifications of the prototype.

4.2 Code Quality

The code implementation successfully implements all the documented functionalities with proper error handling.

Testing coverage is complete and covers all necessary edge cases.

5 Use of Generative AI

During the creation of this document, we used generative artificial intelligence tools, specifically Large Language Models (LLMs).

We used the tools for the following purposes:

Stylistic improvements: We provided drafts of sections of the work as input and asked for suggestions to improve sentence structure, clarity of exposition, and the precision of technical terminology.

Translation assistance: Since English is not our native language, we used the LLMs to verify the correctness of the translation from Italian, in particular to check grammar and syntax.

All outputs generated by the AI tools were reviewed by both group members to ensure quality, consistency, and correctness.

6 Effort Spent

This section includes the table of hours worked by each group member to complete this document.

Member of group	Effort spent
Raimondi Filippo	3 <i>h</i>
Sala Matteo	3 <i>h</i>
Total	6 h

Table 1: Effort Spent on ATD

References

- [1] A.Y. 2025-2026 Software Engineering 2 - I&T assignment goal, schedule, and rules.
- [2] A.Y. 2025-2026 Software Engineering 2 - Requirement Engineering and Design Project: goal, schedule, and rules.
- [3] Best Bike Path (BBP) Design Document (DD), version 2.
- [4] Best Bike Path (BBP) Implementation and Test Deliverable Document (ITD), version 1.
- [5] Best Bike Path (BBP) Requirements Analysis And Specification Document (RASD), version 4.